

Informe de Auditoría del Sistema Web

Integrantes del grupo:

Jorge Illera Rivera

Iker Argulo Galán

Oscar Lijerón Pérez

Bruno Izaguirre Martínez de Marañón

María Bogajo

Ander Javier Corral

Fecha:

24/11/2024

Índice de Contenidos

1	Introducción	3
2	Metodología	3
3	Vulnerabilidades	3
3.1	Rotura de control de acceso	3
3.1.1	Ausencia de Tokens Anti-CSRF	3
3.1.2	Exposición y modificación de datos en la URL	4
3.1.3	Soluciones implementadas	4
3.2	Fallos criptográficos	6
3.2.1	Soluciones implementadas	6
3.3	Inyección	7
3.3.1	Cross-Site Scripting (XSS)	8
3.3.2	Inyección SQL	8
3.3.3	Soluciones implementadas	9
3.4	Diseño inseguro	10
3.4.1	Soluciones implementadas	10
3.5	Configuración de seguridad insuficiente	11
3.5.1	Credenciales por defecto	11
3.5.2	Soluciones implementadas	12
3.6	Componentes vulnerables y obsoletos	12
3.6.1	Análisis de componentes	12
3.6.2	Soluciones implementadas	14
3.7	Fallos de identificación y autenticación	14
3.7.1	Soluciones implementadas	14
3.8	Fallos en la integridad de datos y software	15
3.9	Fallos en la monitorización de la seguridad	15
3.9.1	Soluciones implementadas	16
4	Conclusiones finales	16
5	Bibliografía	16
6	Apéndices	19

Índice de Figuras

1	Error de rotura de control de acceso detectado por ZAP.	3
2	Errores de inyección detectados por ZAP.	8
3	Desplazamiento del botón Juegos.	11

1 Introducción

Este informe detalla la auditoría de seguridad del sistema web utilizando OWASP ZAP, con el objetivo de identificar y mitigar vulnerabilidades.

2 Metodología

La auditoría se realizó mediante un análisis de ZAP, que generó un informe indicando las diferentes vulnerabilidades en términos de seguridad web de nuestro sistema. Sin embargo, también se han tratado ciertas vulnerabilidades que no se detectaron en la auditoría, pero que se han considerado necesarias para garantizar la seguridad.

3 Vulnerabilidades

3.1 Rotura de control de acceso

¿Qué es una rotura de control de acceso?

Una rotura del control de acceso ocurre cuando un sistema permite a usuarios acceder a datos, funciones o recursos para los que no tienen autorización, debido a fallos en la implementación de los permisos. Esto puede resultar en accesos no autorizados, modificaciones indebidas de datos o acciones privilegiadas que deberían estar restringidas.

En esta sección se han identificado dos amenazas. La primera, que ha sido detectado por ZAP, es la Ausencia de Tokens Anti-CSRF, como se puede observar en la Figura 1. Y la segunda, es la posibilidad de modificar la URL y acceder a cuentas de usuario ajenas, permitiendo ver información confidencial y modificarla.



Figura 1: Error de rotura de control de acceso detectado por ZAP.

3.1.1 Ausencia de Tokens Anti-CSRF

.

Causa de la Vulnerabilidad

La vulnerabilidad se originaba en los archivos `login.php`, `register.php`, `add_item.php` y `modify_user.php` y fue causada por la falta de la generación del token CSRF en pestañas sensibles en las que hay que rellenar un formulario.

Pruebas y comportamiento observado

El error identificado fue detectado por OWASP ZAP al capturar la solicitud original enviada por el navegador al interactuar con el formulario. Posteriormente, ZAP envió una solicitud idéntica sin incluir campos adicionales (como un token¹ anti-CSRF), y esta fue aceptada por el servidor como si se tratara de una solicitud legítima.

3.1.2 Exposición y modificación de datos en la URL

Causa de la Vulnerabilidad

Se detectó que los identificadores sensibles, como los IDs de usuarios y juegos, se transmitían en texto plano a través de las URLs en archivos como `items.php` y `show_item.php`. Esto exponía información crítica que podía ser utilizada para acceder o manipular recursos restringidos.

Pruebas y comportamiento observado

La vulnerabilidad fue identificada mediante un análisis manual del tráfico HTTP y de las URLs generadas por el sistema. Este análisis reveló que identificadores sensibles se transmitían en texto plano, exponiendo potencialmente datos críticos a ataques de enumeración de IDs o manipulación maliciosa.

3.1.3 Soluciones implementadas

Para minimizar la utilización de CORS², se ha integrado el uso de tokens CSRF³ con el fin de implementar mecanismos de acceso únicos.

• Integración de tokens CSRF

Se ha implementado la creación de los tokens en archivos donde se manejan formularios o se ingresan los identificadores y contraseñas de los usuarios. Esto incluye archivos como `login.php`, `register.php`, `add_item.php` y `modify_user.php`.

1. Generación del token CSRF

Si no existe ya un token CSRF en la sesión `$_SESSION['csrf_token']`, se genera

¹Pequeño fragmento de datos generado y utilizado en sistemas informáticos para representar de manera segura una identidad, un estado, o una sesión de comunicación entre partes.

²CORS: Mecanismo que permite que los recursos de un servidor sean accesibles desde diferentes dominios de manera controlada.

³CSRF: Códigos únicos generados para verificar que una solicitud proviene de una fuente legítima y no de un atacante.

uno nuevo usando `bin2hex(random_bytes(32))`. Con esto se asegura que cada sesión tiene un token único y difícil de adivinar.

2. Uso del token en el formulario

En el formulario HTML⁴ (HyperText Markup Language)(`login.html`), este token se debe enviar como un campo oculto. Esto permite que el servidor valide que el formulario fue enviado desde la misma aplicación.

3. Validación del token en el servidor

Cuando el formulario se envía, el código verifica si el token recibido (`$_POST['csrf-token']`) coincide con el que está almacenado en la sesión. Si no coinciden o el token no existe, la solicitud se considera no válida y se bloquea (`die("Error de validación CSRF");`).

4. Eliminación del token

Una vez que el usuario inicia sesión correctamente, el token se elimina usando el comando `unset($_SESSION['csrf-token'])`. Esto asegura que no pueda ser reutilizado.

• Cifrado de URL

Se ha implementado el cifrado de identificadores sensibles en las URL para garantizar que los datos críticos no se expongan en texto plano. Esto incluye ID de usuarios y otros identificadores que podrían ser utilizados por atacantes para realizar accesos no autorizados.

1. Cifrado de identificadores

Los identificadores sensibles, como el ID de usuario, son cifrados utilizando el algoritmo AES-256-CBC. Este método asegura que los datos cifrados sean ilegibles sin la clave de descifrado correspondiente. El código se genera usando `openssl_encrypt()` y se almacena en formato `base64`.

2. Descifrado de URL en el servidor

Cuando el servidor recibe una URL que contiene un identificador cifrado, este es descifrado mediante el algoritmo AES-256-CBC y la misma clave utilizada para cifrarlo. Si el descifrado falla o el identificador no es válido, se bloquea el acceso al recurso solicitado.

3. Uso del cifrado en enlaces

En archivos como `login.php`, `items.php` y `show_user.php`, los identificadores que se incluyen en los enlaces son cifrados antes de enviarse al cliente. Esto asegura que el cliente nunca vea un identificador en texto plano en la URL.

4. Validación de identificadores cifrados

Los identificadores descifrados son validados en el servidor para garantizar que sean valores enteros válidos (`intval($id)`). Si no lo son, la solicitud se rechaza de inmediato para prevenir posibles ataques.

Resultados de la segunda auditoría

⁴Lenguaje de marcado utilizado para estructurar y organizar el contenido en las páginas web.

Después de implementar las soluciones descritas, se realizó un segundo análisis con OWASP ZAP. Los resultados fueron positivos: no se detectaron vulnerabilidades por falta de token CSRF.

Para el caso del cifrado de URLs, un análisis manual confirmó que los identificadores sensibles ya no se transmiten en texto plano. Las pruebas de uso del sistema mostraron que las URLs generadas contenían identificadores cifrados, que eran correctamente descifrados en el servidor antes de cualquier operación.

Conclusiones

La integración de tokens CSRF resolvió las vulnerabilidades detectadas en el control de acceso, garantizando que las solicitudes al servidor sean legítimas y protegidas contra ataques. Esta implementación fortaleció la seguridad en archivos clave como `login.php` y `register.php`, eliminando las fallas detectadas en la primera auditoría. La segunda evaluación con OWASP ZAP confirmó la efectividad de estas medidas, mejorando significativamente la protección del sistema.

Adicionalmente, el cifrado de URLs eliminó la exposición de identificadores sensibles en texto plano, mitigando posibles ataques relacionados con la enumeración o manipulación de IDs. Esto incrementó la seguridad de archivos como `items.php` y `show_item.php`, contribuyendo a un acceso más seguro a los recursos del sistema.

3.2 Fallos criptográficos

En esta sección se comentarán los fallos de seguridad web relacionados con la criptografía⁵ (o la falta de esta) que causan la exposición de datos sensibles[13].

Se han identificado y solucionado dos fallos criptográficos. El primer fallo consiste en almacenar la información de la base de datos sin cifrar, utilizando únicamente el cifrado de datos en reposo incorporado por la base de datos por defecto, el cual se descifra directamente al realizar una consulta[16]. El segundo fallo detectado es almacenar las contraseñas en texto plano.

3.2.1 Soluciones implementadas

Para subsanar el primer error se ha decidido cifrar la información que se almacena en la base de datos. El algoritmo de cifrado simétrico⁶ que se ha usado es el AES (Advanced Encryption Standar) de 256 bits de longitud, puesto que este algoritmo es considerado seguro a día de hoy[15].

El uso de este algoritmo requiere una clave segura para cifrar y descifrar los datos. Para ello se ha usado una función[6] de Python⁷ que genera la clave usando la librería `secrets`, que es una librería que genera números aleatorios con la suficiente entropía para que la clave generada

⁵Práctica que consiste en proteger información mediante el uso de algoritmos codificados, Hashes y firmas.

⁶Método de cifrado que usa la misma clave para cifrar y descifrar.

⁷Lenguaje de programación orientado a objetos usado principalmente para el desarrollo web y de aplicaciones informáticas.

pueda ser considerada criptográficamente fuerte[5]. Una vez generada la clave de secreta se guarda como una variable de entorno⁸ en un archivo `.env` llamado `claveCifrado.env` para evitar comprometerla. Los datos cifrados de la tabla Juegos son el nombre, género, fecha de lanzamiento, valoración y precio. Mientras que los datos cifrados de la tabla Usuarios son el nombre, teléfono, DNI y fecha de nacimiento. Sin embargo, como al cifrar los datos su número de caracteres aumenta, se han tenido que cambiar los tipos de dato y la longitud de los campos de la base de datos para poder almacenarlos.

Para solventar el segundo error se ha decidido usar un algoritmo resumen para generar el resumen criptográfico o Hash⁹ de la contraseña en vez de guardarla como texto plano. Para ello se ha usado la función `password_hash` a la que se le pasa como parámetros la contraseña a resumir y una constante de PHP¹⁰ (Hypertext Preprocessor) `PASSWORD_BCRYPT` que indica que queremos usar el algoritmo Bcrypt[12]. Bcrypt usa sales¹¹ para mayor seguridad ya que hace que dos contraseñas iguales de usuarios diferentes generen resúmenes criptográficos distintos, por lo que se hace mucho mas difícil precalcularlos.

Conclusiones

Subsanar los fallos criptográficos es una tarea fundamental para garantizar la confidencialidad y la confianza en sistemas web. Además, estas vulnerabilidades pueden llevar a la pérdida de confianza de los usuarios y a sanciones legales por no cumplir la Ley de Protección de Datos[18].

Finalmente, se ha conseguido abordar de manera efectiva ambos fallos criptográficos gracias a las medidas implementadas, mejorando significativamente la protección de datos y la fiabilidad del sistema web.

3.3 Inyección

¿Qué es la inyección?

La inyección es una vulnerabilidad de seguridad en la que un atacante introduce datos maliciosos en una aplicación, a menudo mediante campos de entrada o URL, logrando manipular comandos o consultas dentro del sistema. Esto permite ejecutar código no autorizado, acceder a datos sensibles o modificar información sin permisos[1].

En esta sección se han identificado dos amenazas de alto riesgo. Las vulnerabilidades detectadas corresponden a una Inyección SQL y a un ataque de Cross-Site Scripting (XSS) Reflejado, como se puede observar en la Figura 2.

⁸Las variables de entorno son variables disponibles para un programa/aplicación de forma dinámica durante el tiempo de ejecución. El valor de estas variables puede proceder de diversas fuentes: archivos de texto, gestores secretos de terceros, scripts de llamada, etc.[7]

⁹Criptograma que consiste en una sucesión de caracteres alfanuméricos que representan la totalidad del contenido original.

¹⁰PHP es un lenguaje de código abierto especialmente adecuado para el desarrollo web y que puede ser incrustado en HTML.[4]

¹¹Caracteres aleatorios que se le añaden a la contraseña para que sea mas difícil precalcular los resúmenes criptográficos.



Figura 2: Errores de inyección detectados por ZAP.

3.3.1 Cross-Site Scripting (XSS)

La vulnerabilidad CWE-79 corresponde a Cross-Site Scripting (XSS) Reflejado, que permite a un atacante inyectar código malicioso en los parámetros de entrada del sistema, como los campos de formularios o las URL. Cuando un usuario accede a una URL o un formulario que contiene este código inyectado, el navegador del usuario ejecuta dicho código, lo que puede llevar a la manipulación de datos, el robo de cookies, la ejecución de scripts no autorizados, y otros ataques de seguridad.[2]

Causa de la vulnerabilidad

La vulnerabilidad XSS detectada se originaba en el archivo `modify_item.php`, más concretamente en el campo de la fecha de lanzamiento y en otros campos de entrada en el formulario de modificación de un juego. Esto se debía a que los valores de entrada eran reflejados en la respuesta sin la sanitización¹² o validación adecuada, permitiendo a los atacantes inyectar y ejecutar código en el navegador del usuario.

Pruebas y comportamiento observado

Sin la sanitización de entrada OWASP ZAP detectó que ciertos campos permitían la inyección de código en los valores de los formularios. Por ejemplo, en el campo de la fecha, se podía ingresar un valor como `<script>alert(1);</script>`, lo cual provocaba la ejecución de una alerta en el navegador.

3.3.2 Inyección SQL

La vulnerabilidad CWE-89 corresponde a la inyección SQL, donde un atacante es capaz de insertar o manipular consultas SQL a través de los datos de entrada, en este caso, el parámetro

¹²Sanitización se refiere al proceso de limpiar o filtrar los datos de entrada para eliminar cualquier contenido malicioso que pudiera comprometer la seguridad del sistema.[3]

id en la URL.

Causa de la Vulnerabilidad

La vulnerabilidad detectada se originaba en el archivo `show_user.php` y fue causada por la construcción dinámica de consultas SQL sin la adecuada sanitización ni parametrización de los datos proporcionados por el usuario. En este caso, el valor del parámetro `id` se concatenó directamente a la consulta SQL sin protección, lo que permitió la ejecución de código SQL malicioso.

Pruebas y comportamiento observado

En el error encontrado, la inyección fue posible al modificar el parámetro de la siguiente manera: `1750' AND '1'='1' --`. Esto permitió a OWASP ZAP modificar la consulta SQL ejecutada en el servidor, manipulando los resultados devueltos y accediendo a información que no debía ser revelada. En particular, se manipuló la consulta para obtener datos utilizando una condición booleana que siempre era verdadera, `'1'='1'`, y luego se comentó el resto de la consulta utilizando `--`, lo que invalidaba el resto de la consulta SQL.

3.3.3 Soluciones implementadas

Para mitigar la vulnerabilidad de XSS se implementaron las siguientes soluciones:

- **Escapado¹³ de salida:**

Se utilizó la función `htmlspecialchars()` en PHP para escapar caracteres especiales (como `¡, ¿, &, ", ')` en todas las salidas. Esto asegura que el contenido reflejado en el navegador sea tratado como texto y no como código ejecutable.

- **Validación y sanitización de fecha de lanzamiento:**

Se implementó una expresión regular (`preg_match`) para asegurarse de que las fechas ingresadas cumplan con el formato `YYYY-MM-DD`.

- **Sanitización de datos entrantes:**

Para cada valor de entrada recibido a través del formulario, se aplicó `mysqli_real_escape_string()` para evitar que cualquier código especial o malicioso llegue a la base de datos.

- **Validación de Campos Numéricos (rating y price):**

Los valores de `rating` y `price` se convirtieron a tipo `float` usando `floatval()` para asegurarse de que se almacenen y procesan correctamente como valores numéricos.

Para mitigar la vulnerabilidad de Inyección SQL se implementaron las siguientes soluciones:

- **Uso de consultas preparadas:**

Se ha utilizado el enfoque de consultas preparadas con declaraciones `bind_param()`. Esto asegura que los parámetros sean tratados de forma segura, separando los datos del código SQL, y evitando así que cualquier entrada maliciosa pueda modificar la estructura de las consultas.

¹³Escapado se refiere al proceso de transformar caracteres especiales en una entrada válida para que no sean interpretados como parte del código ejecutable.[3]

- **Validación y sanitización de entradas**

Además de las consultas preparadas, se valida y filtra las entradas del usuario para asegurarse de que solo se acepten datos válidos. Por ejemplo, el uso de `intval()` para el parámetro `id` convierte cualquier valor no numérico a un valor seguro (como 0), evitando que se inyecten datos inesperados en las consultas.

Todas estas soluciones también se han implementado en otros archivos que, aunque no presentaban errores en el análisis, tenían vulnerabilidades relacionadas con la inyección que necesitaban ser abordadas.

Resultados de la segunda auditoría

Después de implementar las soluciones descritas, se realizó un segundo análisis con OWASP ZAP. Los resultados fueron positivos: no se detectaron vulnerabilidades de XSS ni otros problemas de inyección.

Conclusiones

Ambas vulnerabilidades fueron una amenaza crítica para el sistema, ya que podía permitir a los atacantes ejecutar código malicioso en el navegador de los usuarios o insertar consultas SQL a través de los datos de entrada. Gracias a las soluciones implementadas, las vulnerabilidades se han mitigado de manera efectiva. El sistema web ahora cumple con un estándar de seguridad más alto, protegiendo los datos frente a posibles ataques de inyección de código.

3.4 Diseño inseguro

¿Que significa diseño inseguro?

Una página web tiene un diseño inseguro cuando su estructura, funcionalidades, o configuración comprometen la seguridad de los usuarios o de sus datos. Esto puede deberse a malas prácticas en el desarrollo, falta de medidas de seguridad adecuadas o errores en la implementación.

En esta sección se ha identificado una vulnerabilidad principal, la ubicación de la funcionalidad relacionada con los juegos. Esta se situaba en la página inicial, haciendo que cualquier usuario de la página, sin ni siquiera haberse registrado, pudiera ver, añadir, modificar o eliminar algún juego de la base de datos. .

3.4.1 Soluciones implementadas

Para mitigar la vulnerabilidad encontrada se ha implementado la siguiente solución:

Desplazamiento del botón para ver/añadir/modificar juegos:

Se ha desplazado el botón de la página inicial a una página posterior después de haber iniciado sesión, como se puede apreciar en la Figura 3. Con esto se consigue evitar que usuarios sin haberse registrado e iniciado sesión puedan llegar a saturar la base de datos y que esta deje de responder.



Figura 3: Desplazamiento del botón Juegos.

Conclusiones

Gracias a las soluciones implementadas, se ha mejorado significativamente la seguridad de la plataforma al restringir funcionalidades sensibles, como la visualización, modificación y adición de juegos, únicamente a usuarios registrados. Este diseño evita que usuarios no autenticados puedan saturar la base de datos mediante solicitudes maliciosas o excesivas, reduciendo el riesgo de ataques como el Denial of Service (DoS). Además también refuerza su arquitectura contra malas prácticas de diseño inseguro, promoviendo un entorno más robusto y confiable.

3.5 Configuración de seguridad insuficiente

¿Qué es la configuración de seguridad insuficiente?

La configuración de seguridad insuficiente ocurre cuando un sistema o aplicación utiliza configuraciones predeterminadas, credenciales simples o no establece medidas de protección adecuadas contra posibles amenazas. Esto puede incluir la falta de restricciones de acceso, configuraciones inadecuadas de servicios, y uso de contraseñas débiles o predeterminadas [17].

En esta sección se ha detectado una vulnerabilidad principal, el uso de credenciales (usuario y contraseña) por defecto o extremadamente simples.

3.5.1 Credenciales por defecto

Esta vulnerabilidad se veía reflejada en dos de nuestros servicios, MariaDB y phpMyAdmin.

- Credenciales de MariaDB:
Usuario: admin
Contraseña: test
- Al igual que en phpMyAdmin:
Usuario: admin
Contraseña: test

Estas credenciales son extremadamente simples y no cumplen con las políticas básicas de seguridad para la autenticación. Y además, dejan expuestos servicios críticos, aumentando significativamente el riesgo de ataques como la inyección SQL, robo de datos y escalamiento de privilegios.

3.5.2 Soluciones implementadas

Para mitigar este problema, se han modificado tanto los usuarios como las contraseñas de ambos servicios por unos valores más seguros. Las nuevas contraseñas cuentan con más de 10 dígitos entre los que se incluyen combinaciones de mayúsculas, minúsculas, números y caracteres especiales.

Conclusiones

El uso de credenciales predeterminadas o débiles representaba un riesgo crítico para la seguridad del sistema, ya que las anteriores eran especialmente conocidas y fáciles de explotar por atacantes. Esto facilitaba y posibilitaba el acceso no autorizado a servicios clave como MariaDB y phpMyAdmin, comprometiendo la integridad y confidencialidad de los datos.

3.6 Componentes vulnerables y obsoletos

¿En que afecta a la seguridad no tener los componentes actualizados?

Los componentes vulnerables y obsoletos son una de las principales causas de ataques en aplicaciones web, ya que los atacantes suelen explotar vulnerabilidades conocidas en versiones antiguas de software. Este apartado analiza los componentes del sistema para identificar y mitigar riesgos asociados con componentes desactualizados en el sistema web desplegado.

Identificación de componentes

El sistema web en cuestión depende de múltiples componentes:

- **Linux:** como sistema operativo base.
- **Apache:** como servidor web.
- **MariaDB:** como sistema de base de datos.
- **PHP:** como lenguaje de backend.

3.6.1 Análisis de componentes

- **Apache:**
 - **Comandos para la detección de versión:**
`docker exec -it docker-lamp-web_1 bash`
`apache2 -v`
 - **Versión:** 2.4.25

- **Debilidades conocidas:**
 - * **CVE¹⁴-2017-15710:** Permite a un atacante remoto realizar un ataque DoS (Denial of Service) utilizando una solicitud HTTP especialmente manipulada.
 - * **CVE-2017-9798:** Permite la ejecución remota de código a través de una configuración incorrecta del servidor.
 - * **CVE-2017-3169:** Desbordamiento de búfer en la función `mod_ssl` que podría permitir a un atacante ejecutar código arbitrario.
- **MariaDB:**
 - **Comandos para la detección de versión:**
`docker exec -it docker-lamp-db_1 mysql -V`
 - **Versión:** 10.8.2
 - **Debilidades conocidas:**
 - * **CVE-2022-27480:** Explotación de binlogs para realizar escaladas de privilegios.
 - * **CVE-2022-27376:** Vulnerabilidad en conexiones TLS que permite ataques de intermediario.
 - * **Otras:** Vulnerabilidades generales en versiones inferiores a 10.9, relacionadas con denegaciones de servicio (DoS).
- **PHP:**
 - **Comandos para la detección de versión:**
`docker exec -it docker-lamp-web_1 bash`
`php -v`
 - **Versión:** 7.2.2
 - **Debilidades conocidas:**
 - * **CVE-2019-11043:** RCE explotable en configuraciones de Nginx+PHP-fpm (aplicable a sistemas similares).
 - * **CCVE-2018-7584:** Ejecución de código arbitrario mediante `unserialize()`.
 - * **Obsolescencia:** PHP 7.2 alcanzó el fin de soporte en 2020 y no recibe actualizaciones.
- **phpMyAdmin:**
 - **Versión:** latest
 - **Debilidades conocidas:**
 - * **latest:** El uso de latest para especificar la versión de un contenedor Docker, como en el caso de phpMyAdmin (especificando `phpmyadmin/phpmyadmin:latest`), hace referencia a la última versión disponible de una imagen Docker. Las actualizaciones automáticas pueden traer vulnerabilidades o no estar lo suficientemente testeadas, lo que pone en riesgo el sistema. Además se podría perder control sobre qué versiones tienen parches de seguridad importantes.

¹⁴El término CVE (Common Vulnerabilities and Exposures) es un sistema de identificación estándar utilizado para referirse a vulnerabilidades de seguridad conocidas en software y hardware.

3.6.2 Soluciones implementadas

- **Dockerfile:** Versión de PHP con Apache actualizado (php:8.1-apache).
- **Docker-compose:**
 - Versión de MariaDB actualizada a la versión 10.9 (image: mariadb:10.9).
 - Versión de phpMyAdmin actualizada a la versión 5.1.1 (image: phpmyadmin/phpmyadmin:5.1.1).

Conclusiones

Mantener un enfoque proactivo en la actualización y monitoreo de los componentes es clave para proteger el sistema frente a nuevas amenazas. Además evitar el uso de la etiquetas como latest es algo a tener en cuenta en futuras versiones o proyectos, debido al compromiso en el que pone al sistema en cuanto a seguridad y estabilidad. Las medidas que se han tomado eliminan vulnerabilidades conocidas y alinean los componentes con buenas prácticas de seguridad.

3.7 Fallos de identificación y autenticación

¿Qué es un fallo de identificación y autenticación?

Un fallo de identificación ocurre cuando el sistema no reconoce quién dices ser. Esto puede deberse a varias razones, como ingresar datos incorrectos o inexistentes, que el sistema no busque correctamente el usuario en la base de datos, o incluso por el uso de mensajes de error ambiguos o confusos que no ayudan al usuario a identificar el problema.

¿Cómo ocurre un ataque por fallos de identificación o autenticación?

Esta vulnerabilidad se da cuando un atacante obtiene una lista de usuarios y contraseñas que pueden ser válidas en el sistema, y luego intenta acceder a las cuentas mediante un proceso automatizado. Un ejemplo común es el ataque de Credential stuffing. En este ataque, el atacante utiliza credenciales filtradas de otros servicios donde los usuarios han utilizado las mismas contraseñas, y las prueba en el sistema objetivo para obtener acceso a sus cuentas. Si el sistema no implementa medidas adicionales de seguridad, como la autenticación multifactor¹⁵ o límites de intentos de acceso fallidos, el atacante puede obtener acceso a las cuentas de usuario con relativa facilidad.

3.7.1 Soluciones implementadas

Para reducir los riesgos de los errores en la identificación y autenticación se ha implementado la siguiente solución:

¹⁵Autenticación multifactor se refiere a un método de autenticación que requiere que el usuario proporcione dos o más factores de verificación diferentes para acceder a un sistema

Bloqueo temporal de usuarios por Logins¹⁶ fallidos:

Se ha implementado un sistema de bloqueo temporal con ayuda de un archivo `.log`¹⁷ (la creación de este archivo se menciona en el apartado 3.9). Este permite registrar aquellos usuarios que han fallado 5 veces el inicio de sesión en los últimos 10 minutos. Dicha cuenta se bloquea por 10 minutos y si el usuario intenta hacer nuevamente un Login, se le mostrará el tiempo que le falta para que pueda volver a realizar otros 5 intentos. Esto se logra gestionar en el archivo `log` con la función: `function checkFailedAttempts(email, log_file, max_attempts = 5, lock_time = 600).`

Conclusiones

Con estas medidas, el sistema web cumple con estándares más altos de seguridad, evitando ataques de fuerza bruta o de Credential Stuffing en cierta medida. Sin embargo, es importante considerar la implementación de mecanismos como autenticación multifactor y sistemas de alerta para accesos sospechosos.

3.8 Fallos en la integridad de datos y software

¿Qué son los fallos en la integridad de datos y software?

Se refieren a vulnerabilidades que surgen cuando un sistema, aplicación o infraestructura no cuenta con mecanismos adecuados para garantizar que el software y los datos se mantengan libres de manipulaciones no autorizadas. Estas fallas pueden manifestarse en diversas formas, como el uso de componentes no confiables, actualizaciones automáticas inseguras o ataques a la cadena de suministro. El problema radica en que la falta de controles para verificar la autenticidad y la integridad permite a actores maliciosos inyectar código o alterar datos, comprometiendo el sistema y su confiabilidad [19].

En esta sección no se ha encontrado ninguna vulnerabilidad principal, pero una causa común es el uso de bibliotecas o software no confiables. Esto es porque las dependencias sin verificar pueden contener vulnerabilidades o código malicioso.

Por ello, cabe mencionar que en el desarrollo del sistema actual, se utiliza la librería `secrets` de Python para la generación de datos aleatorios seguros. Pero esta librería, al ser parte de la biblioteca estándar de Python, es confiable y no introduce riesgos de integridad ya que sigue prácticas sólidas de generación de números pseudoaleatorios seguros para criptografía [5].

3.9 Fallos en la monitorización de la seguridad

¿Qué son los fallos en la monitorización de la seguridad?

Se refieren a la ausencia o insuficiencia de registros (logs) y sistemas de monitorización necesarios para identificar, alertar y responder a incidentes de seguridad en un sistema. Estas fallas dificultan la detección temprana de actividades sospechosas, lo que puede prolongar el tiempo

¹⁶Proceso mediante el cual un usuario se identifica y autentica en un sistema, servicio o aplicación.

¹⁷Un `.log` es un archivo de texto que se utiliza para registrar eventos, actividades o mensajes de un sistema, programa o aplicación

en el que un ataque permanece sin ser detectado. Además, la falta de registros adecuados complica la investigación posterior de incidentes y la comprensión de su alcance. [20].

En esta sección, se contaba con una vulnerabilidad principal, se carecía de un registro de logs del sistema.

3.9.1 Soluciones implementadas

Registro de todos los intentos de sesiones (logs):

Se ha utilizado un archivo .log para poder gestionar y observar los intentos de sesiones y el por qué de los errores. En el archivo .log se registra lo siguiente:

1. Fecha y hora en formato yyyy-mm-dd hh:mm:ss.
2. SUCCESS o FAILED.
3. Email registrado o no en la base de datos.
4. IP¹⁸.

Conclusiones

Al registrar información clave como la fecha, hora, estado del intento (SUCCESS o FAILED), y direcciones IP involucradas, se facilita el análisis forense en caso de incidentes de seguridad. Esto también sienta las bases para implementar medidas de seguridad adicionales, como el bloqueo automático de IPs sospechosas o la aplicación de políticas más estrictas para la gestión de contraseñas.

4 Conclusiones finales

-puntos importantes: se han subsanado 2 vulnerabilidades de riesgo alto y 1 de riesgo medio la mayoría de cosas solucionadas no se reflejaban como un problema en ZAP han quedado dos vulnerabilidades de riesgo medio sin resolver pero implican la modificación de la imagen web

5 Bibliografía

References

- [1] owasp.org
Injection
<https://owasp.org/www-community/Injection> (accedido 11/07/2024).
- [2] owasp.org
Cross-Site Scripting (XSS)
<https://owasp.org/www-community/attacks/xss/> (accedido 11/07/2024).

¹⁸IP: Identificador único que se asigna a cada dispositivo conectado a una red

- [3] cwe.mitre.org
Sanitización y escapeo
<https://owasp.org/www-community/attacks/xss/> (accedido 11/07/2024).
- [4] php.net
¿Qué es PHP?
<https://www.php.net/manual/es/intro-what-is.php> (accedido 11/07/2024).
- [5] python.org
¿Qué es la librería secrets?
<https://docs.python.org/es/3/library/secrets.html> (accedido 21/11/2024).
- [6] Python Code: Generación de Claves de Cifrado
Este código en Python se utiliza para generar claves criptográficamente seguras usando la librería secrets

```
import secrets

def generar_clave_encriptacion(longitud=32):

    clave = secrets.token_bytes(longitud) # Devuelve una cadena
    de bytes aleatorios

    return clave.hex()

clave_encriptacion = generar_clave_encriptacion()

print(f"Clave de encriptacion generada: {clave_encriptacion}")
```

- [7] kinsta.com
¿Que es una variable de entorno?
<https://kinsta.com/es/base-de-conocimiento/variables-de-entorno/>
(accedido 15/11/2024).
- [8] OWASP Foundation
¿Qué son los fallos de identificación y autenticación?
https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/ (accedido 23/11/2024).
- [9] SANS Institute Blog
¿Qué es la autenticación multifactor?
<https://www.sans.org/blog/deploying-multi-factor-authentication-the-what>
(accedido 23/11/2024).

- [10] OWASP Foundation
¿Qué es el Credential Stuffing?
https://owasp.org/www-community/attacks/Credential_stuffing (accedido 23/11/2024).
- [11] Postel, J. (Editor)
¿Qué es la IP?
<https://www.rfc-editor.org/rfc/rfc791.html> (accedido 23/11/2024).
- [12] skysnag.com
¿Que es Bcrypt?
<https://www.skysnag.com/es/blog/what-is-bcrypt/> (accedido 15/11/2024).
- [13] owasp.org
¿Que son los fallos criptográficos?
https://owasp.org/Top10/es/A02_2021-Cryptographic_Failures/ (accedido 16/11/2024).
- [14] aws.amazon.com
¿Que es la criptografía?
<https://aws.amazon.com/es/what-is/cryptography/> (accedido 16/11/2024).
- [15] msmk.university
¿Qué es AES? ¿Es seguro?
<https://msmk.university/advanced-encryption-standard-aes/> (accedido 16/11/2024).
- [16] mariadb-com
Limitaciones e información sobre el cifrado de MariaDB
https://mariadb-com.translate.google.com/translate/kb/en/data-at-rest-encryption-overview/?x_tr_sl=en&x_tr_tl=es&x_tr_hl=es&x_tr_pto=rq (accedido 21/11/2024).
- [17] owasp.org
Configuración de seguridad insuficiente
https://owasp.org/Top10/A05_2021-Security_Misconfiguration/ (accedido 22/11/2024).
- [18] boe.es
Ley de protección de datos
<https://www.boe.es/buscar/act.php?id=BOE-A-2018-16673> (accedido 23/11/2024).
- [19] owasp.org
Fallos en la integridad de datos y software
https://owasp.org/Top10/A08_2021-Software_and_Data_Integrity_Failures/ (accedido 23/11/2024).

[20] owasp.org

Fallos en la monitorización de la seguridad.

https://owasp.org/Top10/A09_2021-Security_Logging_and_Monitoring_Failures/ (accedido 23/11/2024).

6 Apéndices